Commercial-license-first LiDAR generation PoC

One diffusion model • One real sensor dataset • Fine-tuning architecture and code-generation specification

Research cutoff: 2 November 2026 | Status: MIT source and MIT dataset stated by publishers; pretrained model weights NOT cleared

Executive decision

Use LidarDM as the sole generative diffusion-model codebase, and Pandar128 Lane Line Detection as the sole real LiDAR dataset. Both publicly display MIT licenses. For a strictly clean proprietary-commercial path, DO NOT use LidarDM's KITTI-360/Waymo-pretrained checkpoints; initialize and train your own model weights exclusively on approved data. This pairing is a starting architecture, not a turn-key compatible pretrained model. The dataset is driving LiDAR, with limited diversity relative to a general scene generator.

Artifact	Selected sole name	Published license	Commercial gate
Generative model source	LidarDM	MIT (GitHub)	Use code; train clean weights
Real recorded data	Pandar128 Lane Line Detection	MIT (GitHub + Hugging Face)	Pin dataset revision + license + provenance
Existing released checkpoints	LidarDM KITTI-360 / Waymo weights	Not established as MIT	EXCLUDE

Publisher evidence & exact resources

Model source: https://github.com/vzyrianov/LidarDM — GitHub marks code MIT. The repository explicitly trains on KITTI-360 and Waymo Open Dataset; it offers pretrained weights associated with these datasets and describes access conditions for Waymo weights.

Real dataset: https://huggingface.co/datasets/filipberanek/pandar128-lane-line-detection — dataset card marks MIT; https://github.com/filipberanek/lane_lines_dataset — repository identifies its dataset as released under MIT. Its 29 traces include a Hesai Pandar128, camera, odometry, calibration, LiDAR scan NPZ archives and train/validation/test archives. Approximate size: 127 GB train, 39 GB validation, 35 GB test; source is focused on lanes.

License classification: exact scope

Asset	Decision	Why
LidarDM source code	ACCEPT for source reuse	MIT repository license; preserve notices
Pandar128 published files	PROVISIONALLY ACCEPT	Publisher labels dataset MIT; preserve copies and rights

		records
LidarDM downloadable pretrained weights	REJECT for strict MIT-only workflow	No independent blanket MIT grant verified for checkpoints; trained with third-party data
Upstream dependencies & assets	PENDING audit	Individual component licenses may differ
Generated output weights	Candidate proprietary artifact	Only if all inputs, weights and dependencies pass audit

Important: “MIT” labeling is publisher evidence, not a legal warranty. Confirm license file covers downloadable dataset archives, third-party contributions, privacy/collection rights, and any downstream redistribution conditions before commercialization. Code and data are separate legal assets.

Technical scope and architecture

Goal: fine-tune/train a diffusion-based LiDAR generator on real Pandar128 point-cloud sequences to generate realistic sensor observations, preserving scene consistency when conditioned on a synthetic scene. A clean training run from random initialization is the required first version; LoRA fine-tuning is an optional second experiment only after obtaining an independently cleared base checkpoint.

Pipeline: Pandar128 scan NPZ → data parser → sensor-frame normalization → LiDAR range-image / occupancy representation → LidarDM training adapter → generative model → raydrop/intensity realism module → synthetic samples → held-out real-data comparison.

Core mathematical formulation

Let x0 be a real LiDAR representation, c=(sensor intrinsics, beam elevations, pose or scene conditioning), t a diffusion step, and $\varepsilon \sim N(0,I)$. For VP-style diffusion, $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\varepsilon$.

Noise prediction loss: $L_{diff} = E_{t,x0,c,\varepsilon}[\,|| \varepsilon - \varepsilon_\varphi(x_t,t,c) ||^2]$. Handle missing/invalid returns with an explicit mask; do not treat missing returns as valid zero-range points.

When scene conditioning s exists: $L_{total} = L_{diff} + \lambda_{geom} L_{geom} + \lambda_{drop} L_{drop} + \lambda_{temp} L_{temp}$. Geometry loss evaluates permissible surface deviations; raydrop loss compares valid-return statistics; temporal loss measures sequence consistency. Conditioning may require adapting LidarDM loaders and architecture; this is not claimed as an existing out-of-the-box interface.

Distribution validation uses held-out real sequences: (i) range × elevation conditional Wasserstein distances, (ii) return-rate curves, (iii) per-ring intensity distributions where available, (iv) inter-frame persistence, (v) geometry preservation, (vi) downstream perception transfer if a suitable labeled task can be evaluated. Low distribution distance does not prove downstream utility.

Implementation repository layout

```
poc/
  LICENSES/  # pinned evidence, licenses, SBOM
  configs/{data.yaml,train.yaml,eval.yaml}
  src/data/{pandar_reader.py,scan_to_range.py,split_manifest.py}
  src/models/{lidardm_adapter.py,masking.py}
  src/train.py
```

src/eval/{domain_metrics.py,temporal_metrics.py}
tests/{test_frames.py,test_masks.py,test_split.py,test_license_gate.py}
artifacts/{checkpoints,logs,metrics}

Code-generation contracts

DataLoader output (minimum): {range: float32[B,T,H,W], valid: bool[B,T,H,W], intensity: optional float32, sensor_meta: elevations/azimuths, timestamp_ns: int64, sequence_id: str, pose: optional SE(3)}. Fix coordinate conventions and units in tests. The original NPZ layout must be inspected; do not assume each file has the same internal array key names.

Generator adapter: sample(batch_size, conditioning, seed) -> {range, valid, intensity_optional}; train_step(real_batch) -> {loss, components}; checkpoint must store config, training-data manifest hash, model source commit, runtime dependency lock, and license manifest.

Baseline experiment: train from scratch on real training traces; compare generated held-out statistics against (a) unmodified training resampling and (b) a simple per-ring empirical sensor model. LoRA is not valid without an authorized pretrained checkpoint. RL is out of scope until conventional training and calibration have proven transfer utility.

Executable-style training pseudocode

```
dataset = Pandar128Dataset(manifest="splits/train.json")
loader = DataLoader(dataset, batch_size=8, shuffle=True, num_workers=4)
model = LidarDMAdapter(init_weights="random", sensor="pandar128")
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
for epoch in range(num_epochs):
  for batch in loader:
    z0, mask, cond = project_scan(batch)  # verify scan schema first
    t = sample_diffusion_steps(z0.size(0))
    eps = torch.randn_like(z0)
    zt = alpha_bar[t].sqrt()*z0 + (1-alpha_bar[t]).sqrt()*eps
    eps_hat = model(zt, t, cond)
    loss = masked_mse(eps_hat, eps, mask)
    optimizer.zero_grad(); loss.backward(); optimizer.step()
  save_checkpoint_with_provenance(model, optimizer, manifests)
```

Pseudocode only: it is not a drop-in LidarDM training script. The published architecture uses scene generation plus raydropping components; adapt the relevant modules to the selected range-image/scene representation before running.

Validation, failure criteria and release gates

Gate	Pass criterion
Licensing	MIT license text captured for code and dataset; no forbidden pretraining, weights, or assets
Data	Parse scan schema and calibration successfully; validate units and ring assignment

Splits	Separation by complete driving trace; no frame overlap or near-duplicate leakage
Generative fidelity	Better conditional distribution statistics than simple baselines on unseen traces
Geometry	No unacceptable hallucinated obstacles; preserve sensor/scene constraints
Compute	Stable training and sample generation on agreed hardware budget
Real-world utility	Measured downstream improvement or useful realism gains on held-out sequences

Practical limitations

Pandar128 is a lane-focused, road-driving dataset. It is suitable to prototype LiDAR generation and domain adaptation for one 128-beam scanner, but not enough to claim generalization to other beam geometries, off-road scenes or complex objects. It contains no automotive radar. This report therefore deliberately specifies a LiDAR-only PoC. Strictly MIT/BSD/Apache radar generative checkpoints and real radar datasets were not established as a comparably clear complete stack in this one-model/one-dataset selection.

Source URLs and date

Verified via publicly accessible publisher pages on 2026-11-02:
https://github.com/vzyrianov/LidarDM
https://github.com/filipberanek/lane_lines_dataset
https://huggingface.co/datasets/filipberanek/pandar128-lane-line-detection